PROJET D'ÉVALUATION

Le Mot Mystère

Un jeu de lettres en React, appuyé sur le Wiktionnaire

Développer un jeu de mots à deviner en six essais : grille, validation des mots par l'API du Wiktionnaire, clavier, statistiques, mot du jour partagé, variantes et mode multi-grilles.

Marc Deschamps
Formateur en développement logiciel

Le Mot Mystère

Le contexte

Un média en ligne veut proposer à ses lecteurs un jeu quotidien : **Le Mot Mystère**. Le principe est celui d'un jeu devenu célèbre : trouver un mot de cinq lettres en six essais au maximum. Après chaque proposition, chaque lettre change de couleur :

- **vert** : la lettre est dans le mot, à la bonne place ;
- **jaune** : la lettre est dans le mot, mais à une autre place ;
- **gris** : la lettre n'est pas dans le mot, ou pas autant de fois que proposé.

Les propositions doivent être de vrais mots français : l'application les vérifie auprès du **Wiktionnaire**, le dictionnaire libre de la fondation Wikimedia.

Chaque partie gagnée rapporte des **points**, qui alimentent un tableau des scores :

Mot trouvé en	1 essai	2 essais	3 essais	4 essais	5 essais	6 essais	Perdu
Points	600	500	400	300	200	100	0

Votre mission : **concevoir et développer ce jeu** : une application React fonctionnelle.

Le projet est organisé en **paliers** de difficulté croissante. Vous choisissez jusqu'où aller ; chaque palier validé correspond à une note.

Les contraintes

- L'application est développée avec **React**. La navigation entre les écrans utilise **React Router**.
- **Aucun serveur à développer.** La vérification des mots et leurs définitions proviennent de l'API du Wiktionnaire, appelée depuis l'application. Ce qui doit être conservé entre deux visites (pseudo, tableau des scores, statistiques, partie en cours) est enregistré dans le navigateur.
- Les mots se jouent **sans accents** : « école » se joue `ECOLE`, « cœur » se joue `COEUR`.
- Les paliers 1 à 3 se développent et se présentent **en local**. À partir du palier 4, le mot du jour étant commun à tous les joueurs, le jeu est **mis en ligne sur Vercel**.
- Le travail se fait **en binôme**, sur un dépôt Git partagé. L'historique des commits doit montrer la contribution des deux membres.
- Toute bibliothèque supplémentaire doit être justifiée dans le README.

Les ressources fournies

Le fichier `mots-solutions.json` contient environ 300 mots courants de cinq lettres, écrits avec leurs accents. Les mots mystères sont toujours tirés de cette liste. Les **propositions** du joueur, elles, peuvent être n'importe quel mot français de cinq lettres reconnu par le Wiktionnaire.

L'API du Wiktionnaire

Le Wiktionnaire repose sur le logiciel MediaWiki, qui fournit une API publique, sans clé. Toutes les requêtes passent par la même adresse :

`https://fr.wiktionary.org/w/api.php`

Les paramètres de la requête indiquent l'action voulue. Quelques exemples à explorer :

Besoin	Paramètres
Savoir si une page existe	`action=query&titles=arbre&format=json`
Rechercher des pages	`action=query&list=search&srsearch=ecole&format=json`
Suggestions à partir d'un début de mot	`action=opensearch&search=arbr&format=json`
Connaître les catégories d'une page	`action=query&titles=arbre&prop=categories&format=json`
Obtenir le contenu d'une page	`action=parse&page=arbre&prop=text&format=json`

ATTENTION

Pour qu'une application web puisse appeler cette API depuis le navigateur, chaque requête doit contenir le paramètre `origin=*`. Sans lui, le navigateur bloque la réponse.

À NOTER

Le Wiktionnaire contient des mots de toutes les langues : une page qui existe ne correspond pas forcément à un mot français. À vous de trouver comment le vérifier.

BONNE PRATIQUE

Testez les requêtes directement dans le navigateur avant de coder, en ajoutant `&format=json&origin=*`. Le Wiktionnaire propose aussi un bac à sable qui construit les requêtes pour vous : cherchez « Special:ApiSandbox ».

La fondation Wikimedia demande aux applications de s'identifier : ajoutez à vos requêtes un en-tête `Api-User-Agent` contenant le nom du projet et un moyen de contact.

Les paliers

Un palier n'est validé que si **tous les paliers précédents fonctionnent**. Indiquez dans le README le palier visé et ce qui est terminé.

Palier 1 : le jeu (10/20)

Une page d'accueil et une partie complète, jouable au clavier de l'ordinateur.

Attendus :

- une **page d'accueil** présente le jeu : son nom, une phrase d'explication, un champ pour saisir le **pseudo** du joueur et un bouton pour lancer une partie ; le pseudo est mémorisé pour les visites suivantes ;
- la partie a sa propre adresse, distincte de l'accueil ;
- une grille de 6 lignes de 5 cases ;
- le mot mystère est tiré au hasard dans la liste fournie ;
- les lettres se tapent au clavier, la touche Retour arrière efface, la touche Entrée valide une ligne complète ;
- après validation, chaque lettre prend la bonne couleur, **y compris quand une lettre apparaît plusieurs fois** dans la proposition ou dans le mot ;
- la victoire et la défaite sont détectées ; en cas de défaite, le mot est révélé ;
- à la fin de la partie, le joueur voit les points obtenus, et peut lancer une nouvelle partie ou revenir à l'accueil ;
- la logique du jeu (comparaison d'une proposition avec le mot, état de la partie) est écrite dans des fonctions séparées des composants d'affichage.

Palier 2 : le dictionnaire (12/20)

Seuls les vrais mots sont acceptés.

Attendus :

- avant d'être colorée, chaque proposition est vérifiée auprès du Wiktionnaire : seuls les mots français sont acceptés ;
- un mot refusé ne compte pas comme un essai ; le joueur en est averti et la ligne le montre visuellement ;
- pendant la vérification, le joueur voit que l'application travaille, et ne peut pas valider une seconde fois ;
- si l'API ne répond pas, le joueur en est informé et la partie n'est pas bloquée : décidez d'un comportement raisonnable et justifiez-le dans le README ;
- à la fin de la partie, le mot mystère est affiché avec ses accents et un lien vers sa page du Wiktionnaire.

Palier 3 : le confort de jeu (14/20)

Le jeu devient agréable.

Attendus :

- un **clavier à l'écran**, utilisable à la souris ou au doigt, dont les touches prennent la couleur de ce qu'on sait déjà de chaque lettre ;
- les lettres se **retournent** une à une à la validation, et la ligne **tremble** quand un mot est refusé ;
- un **tableau des scores** : les dix meilleures parties, classées par points, avec pour chacune le pseudo, le mot trouvé, le nombre d'essais, les points et la date ; plusieurs joueurs peuvent se succéder sur le même ordinateur et apparaître dans le même tableau ; le tableau est conservé entre deux visites ;
- un écran de **statistiques** du joueur : parties jouées, pourcentage de victoires, série en cours, meilleure série, total des points, répartition des victoires selon le nombre d'essais ;
- la page d'accueil donne accès à une nouvelle partie, au tableau des scores, aux statistiques et aux règles ;
- un écran de **règles**, avec un exemple illustré des trois couleurs ;
- chaque écran a sa propre adresse.

Palier 4 : le mot du jour (16/20)

Comme dans le jeu original, tout le monde cherche le même mot chaque jour, et le jeu est accessible en ligne.

Attendus :

- l'application est **déployée sur Vercel**, à partir du dépôt Git : chaque envoi sur la branche principale met automatiquement le site à jour ;
- toutes les adresses de l'application fonctionnent en ligne, y compris quand on rafraîchit une page ou qu'on ouvre directement un lien vers une partie ou vers le tableau des scores ;
- un mode « mot du jour » : le même mot pour tous les joueurs, sur tous les ordinateurs, qui change à minuit ; le mode « partie libre » reste disponible ;
- une partie du jour commencée reprend où elle en était si la page est rechargée ; une fois terminée, elle ne peut pas être rejouée ;
- un compte à rebours indique le temps restant avant le prochain mot ;
- le tableau des scores distingue le **classement du mot du jour** (tous les joueurs de l'ordinateur sur le mot du jour, par points puis par temps de résolution) du classement général ;
- le résultat se partage par un texte copié dans le presse-papiers, qui montre les couleurs sans révéler les lettres :

```
Le Mot Mystère 21/10 · 4/6
⬛🟧⬛⬛⬛
⬛🟩🟧⬛⬛
🟩🟩⬛🟩⬛
🟩🟩🟩🟩🟩
```

- au moins **cinq tests automatisés**, dont au moins deux sur la coloration des lettres en double.

Palier 5 : les variantes (18/20)

Le joueur choisit sa difficulté.

Attendus :

- des mots de **6 et 7 lettres** en partie libre, la grille et le nombre d'essais s'adaptant à la longueur. Pour ces longueurs, constituez vous-mêmes une liste de mots solutions, et expliquez dans le README comment vous l'avez obtenue ;
- un **mode difficile** : toute lettre révélée en vert doit rester à sa place et toute lettre jaune doit être réutilisée dans les propositions suivantes ; une proposition qui ne respecte pas ces règles est refusée avec un message précis ;
- les points tiennent compte de la difficulté : × 1,5 pour 6 lettres, × 2 pour 7 lettres, × 1,5 en mode difficile ; le tableau des scores indique la longueur et le mode de chaque partie, et peut être filtré selon ces critères ;
- les réglages (longueur, mode difficile) sont conservés entre deux visites, et la longueur fait partie de l'adresse de la partie.

Palier 6 : les quatre grilles (20/20)

Un mode expert : **quatre mots mystères à trouver en même temps**.

Attendus :

- quatre grilles affichées ensemble, et neuf essais au total ;
- chaque proposition est jouée dans les quatre grilles à la fois ; une grille résolue se fige ;
- le clavier à l'écran montre l'état de chaque lettre pour chacune des quatre grilles ;
- un écran de fin indique les grilles résolues, en combien d'essais, et révèle les mots manqués ;
- les parties à quatre grilles ont leur propre règle de points, que vous définissez et expliquez dans les règles du jeu, et leur propre classement.

Indices

Ces indices ne donnent pas la solution, mais évitent de perdre du temps sur les pièges classiques.

La coloration des lettres en double

- C'est le piège principal du jeu. Avec le mot `ARBRE` et la proposition `ERRER`, toutes les lettres ne peuvent pas être jaunes : il n'y a que deux R et un E dans le mot.
- Une méthode fiable se fait en **deux passages** : marquer d'abord toutes les lettres bien placées, puis seulement attribuer les jaunes, en tenant le compte des lettres du mot encore disponibles.
- Écrivez les tests de cette fonction **avant** de l'utiliser dans l'interface : préparez une dizaine de couples mot / proposition avec le résultat attendu.

Les accents

- Comparez toujours des mots sans accents et en majuscules. JavaScript sait décomposer une lettre accentuée en lettre simple plus accent : cherchez du côté de `normalize("NFD")`.
- La ligature `œ` n'est pas une lettre accentuée : elle doit être remplacée par `oe` à la main.
- Pour vérifier une proposition tapée sans accents (`ECOLE`), cherchez quelle requête du Wiktionnaire retrouve la page « école » ; comparez ensuite les résultats sans accents.

Le clavier de l'ordinateur

- Les touches se captent avec un écouteur d'événement `keydown` sur la fenêtre, ajouté dans un effet et retiré dans sa fonction de nettoyage.
- Attention à la valeur des données lues dans l'écouteur : si l'effet ne se relance pas quand la ligne en cours change, l'écouteur travaille sur une version périmée de l'état. Cherchez pourquoi, et comment `useReducer` peut simplifier les choses.
- Ignorez les touches qui ne sont pas des lettres, et les raccourcis du navigateur (Ctrl, Cmd).

Les animations

- Un retournement de case se fait très bien en CSS, avec une animation et un délai différent pour chaque case de la ligne.
- Pour rejouer une animation de tremblement à chaque refus, il faut retirer puis remettre la classe CSS : un compteur de refus utilisé comme `key` est une piste.

Le même mot pour tous

- `Math.random()` ne peut pas être initialisé : deux ordinateurs obtiendront deux mots différents. Il suffit ici d'un calcul simple à partir de la date (le nombre de jours écoulés depuis une date de départ, par exemple) pour choisir un indice dans la liste.
- Attention au fuseau horaire : le changement de mot doit avoir lieu à minuit, heure française, pour tout le monde.

Le tableau des scores

- Enregistrez chaque partie terminée comme un objet simple (pseudo, mot, essais, points, date, mode) dans une liste ; le classement est **calculé** à partir de cette liste au moment de l'affichage : tri, filtre, limite à dix.
- Le `localStorage` ne stocke que du texte : pensez à `JSON.stringify` et `JSON.parse`, et au cas où la valeur enregistrée est absente ou illisible.
- Deux parties à égalité de points doivent toujours apparaître dans le même ordre : choisissez un second critère de tri.

Le déploiement sur Vercel

- Vercel reconnaît un projet Vite tout seul : la difficulté n'est pas la mise en ligne, mais ce qui se passe ensuite. Testez votre site en ligne en ouvrant directement une adresse comme `/scores`, et en rafraîchissant chaque écran.
- Si une page affiche une erreur 404 en ligne alors qu'elle fonctionne en local, demandez-vous qui répond à cette adresse : le serveur de Vercel, ou React Router ? Cherchez comment configurer les « rewrites » de Vercel pour une application monopage.
- Le `localStorage` est propre à chaque adresse : les scores enregistrés en local n'apparaissent pas sur le site en ligne, et inversement. Ce n'est pas un bug.
- Les paramètres de l'API (adresse du Wiktionnaire, en-tête `Api-User-Agent`) ne changent pas entre le local et le site en ligne.

Le presse-papiers

- `navigator.clipboard.writeText()` copie un texte et renvoie une promesse. Prévoyez un message de confirmation, et le cas où la copie est refusée.

Livrables

1. **Le dépôt Git** de l'application, avec un `README.md` qui précise :
   - le binôme ;
   - le palier visé et l'état de chaque palier ;
   - comment installer, lancer et tester l'application ;
   - les choix techniques importants et les bibliothèques ajoutées ;
   - les difficultés rencontrées et la façon dont vous les avez résolues.
2. **À partir du palier 4, l'adresse du site en ligne sur Vercel**, indiquée en haut du README. C'est sur cette version que le mot du jour est évalué.
3. **Une soutenance de 10 minutes** par binôme :
   - 5 minutes de démonstration, en jouant : en local pour les paliers 1 à 3, sur le site en ligne à partir du palier 4 ;
   - 5 minutes sur un choix technique au choix du binôme et une difficulté rencontrée ;
   - suivies de questions sur le code. Chaque membre doit pouvoir expliquer n'importe quelle partie du projet.

Évaluation

La note dépend du **dernier palier entièrement validé**. Un palier est validé quand tous ses attendus fonctionnent, ainsi que ceux de tous les paliers précédents.

Palier validé	Note
Aucun palier validé	de 0 à 9, selon ce qui fonctionne
Palier 1 : le jeu	10
Palier 2 : le dictionnaire	12
Palier 3 : le confort de jeu	14
Palier 4 : le mot du jour	16
Palier 5 : les variantes	18
Palier 6 : les quatre grilles	20

Palier commencé : si le palier suivant est commencé et qu'une partie de ses attendus fonctionne, la note augmente de 1 point.

Retraits de points, appliqués ensuite à la note du palier :

Manque constaté	Retrait
Erreurs au lint ou au build	1 point
Code mal organisé : logique du jeu mêlée à l'affichage, appels à l'API dispersés, composants géants	1 à 2 points
README incomplet : palier visé, état des paliers, instructions de lancement	1 point
Soutenance : démonstration qui échoue, ou un membre du binôme incapable d'expliquer le code	1 à 2 points

La note finale ne peut pas dépasser 20, ni descendre sous 0.

Exemple : un binôme valide le palier 2 (12), a commencé le palier 3 avec le clavier à l'écran (+1), mais son README n'indique pas comment lancer l'application (-1). Sa note est 12.